These problems cover the Prolog part of the course, Lectures 15 to 21: Prolog basics; logical foundations (first-order logic, definite clauses, Herbrand models, SLD resolution); lists and arithmetic; control (goal order, rule order, substitutions, unification, the abstract interpreter); cuts and negation; generate and test; and type inference and program synthesis in Prolog.

Conventions. SWI-Prolog's rule order (clauses are tried from top to bottom) and goal order (the leftmost goal is solved first) are assumed throughout. When a question asks for *all* answers to a query, list them in the order in which Prolog produces them, as the Jupyter notebook prints them (a query that has no answer is answered `false`); if a query raises an error or does not terminate, say so. The predicates `sum/2`, `len/2`, `is_different/2` and `type/3` are the ones defined in the lectures; they are reproduced wherever a question depends on their exact clauses.

1. Unification and substitutions. Assume the occurs check is enabled throughout this question.
   (a) For each pair of terms say whether they unify.
      (i) `f(X,b)` and `f(a,Y)`
      (ii) `f(X,X)` and `f(a,b)`
      (iii) `g(X,f(Y))` and `g(f(Z),X)`
      (iv) `[X|Y]` and `[1,2,3]`
      (v) `[X,Y|Z]` and `[a]`
      (vi) `p(X,g(X))` and `p(Y,Y)`
      (vii) `h(X,Y,Z)` and `h(Y,Z,f(X))`
      (viii) `f(X,Y)` and `f(Y,X)`
   (b) Let $\theta = \{X/f(Y),\ Z/W\}$ and $\sigma = \{Y/a,\ W/Z,\ X/b\}$.
      (i) Compute $\theta\sigma$.
      (ii) Compute $\sigma\theta$.
      (iii) For $E = \mathtt{p(X,Y,Z,W)}$, compute $E(\theta\sigma)$ and $(E\theta)\sigma$ and check that they agree.
   (c) Which of the following are valid substitutions according to the definition from the lectures? For the invalid ones, say which condition is violated.

$$\{X/Y,\ Y/X\} \qquad \{X/f(X)\} \qquad \{X/a,\ X/b\} \qquad \{a/X\} \qquad \{X/X\} \qquad \{X/Y,\ Z/Y\}$$

   (d) Let $\theta = \{X/Y,\ Y/a\}$. Compute $\theta\theta$. Is $\theta\theta = \theta$? Check your answer against the property "$\theta = \theta\theta$ iff $Dom(\theta) \cap Range(\theta) = \emptyset$".
   (e) Let $s = \mathtt{f(X,g(Y))}$ and $t = \mathtt{f(g(Z),W)}$, and consider

$$\sigma_1 = \{X/g(Z),\ W/g(Y)\} \quad \sigma_2 = \{X/g(a),\ Z/a,\ W/g(b),\ Y/b\} \quad \sigma_3 = \{X/g(Z),\ W/g(Z),\ Y/Z\}$$

   Which of these are unifiers of s and t? Which one is a most general unifier? For each of the others, give a substitution ω such that it equals $\sigma_1\omega$.

2. Logical foundations.
   (a) Write the first-order formula (a definite clause, with all quantifiers explicit) corresponding to each Prolog clause below.
      (i) `likes(X,Y) :- friend(X,Z), likes(Z,Y).`
      (ii) The pair of clauses `happy(X) :- rich(X).` and `happy(X) :- married(X,Y), happy(Y).` Also give a single formula equivalent to the two together.
   (b) Which of the following are definite clauses? (All variables are universally quantified.)
      (i) $p(x) \vee \neg q(x) \vee \neg r(x)$
      (ii) $p(x) \vee q(x) \vee \neg r(x)$
      (iii) $\neg p(x) \vee \neg q(x)$

(iv) $p(a)$
(v) $p(x) \leftarrow q(x) \vee r(x)$

(c) Consider the set of formulas

$$P = \{\ \forall x.\ le(z,x), \qquad \forall x,y.\ le(x,y) \rightarrow le(x,s(y))\ \}.$$

Which of the following interpretations are models of P? For those that are not, give a counterexample.

(i) Domain $\mathbb{N}$; $z \mapsto 0$; $s(x) = x+1$; $le \mapsto \leq$.
(ii) Domain $\mathbb{N}$; $z \mapsto 0$; $s(x) = x+1$; $le \mapsto <$.
(iii) Domain $\mathbb{N}$; $z \mapsto 0$; $s(x) = x$; $le \mapsto \leq$.
(iv) Domain $\mathbb{Z}$; $z \mapsto 0$; $s(x) = x+1$; $le \mapsto \leq$.
(v) Domain $\mathbb{N}$; $z \mapsto 0$; $s(x) = x+1$; le maps to the relation that holds for every pair.

(d) Consider the program

```
edge(a,b).
edge(b,c).
path(X,X).
path(X,Z) :- edge(X,Y), path(Y,Z).
```

(i) How many elements does the Herbrand universe have?
(ii) How many elements does the Herbrand base have?
(iii) List the elements of the least Herbrand model.
(iv) Let M be the least Herbrand model together with the extra atom `path(c,a)`. Is M a Herbrand model of the program? If not, what is the smallest set of atoms that must be added to M to make it one?

3. Consider the following program about course prerequisites (`prereq(X,Y)` means that `X` is a direct prerequisite of `Y`). `is_different` is the predicate from Lecture 19.

```
is_different(A,A) :- !, fail.
is_different(_,_).

prereq(cs1100, cs2200).
prereq(cs1100, cs2700).
prereq(cs2200, cs3100).
prereq(cs2700, cs3100).
prereq(cs2700, cs3300).
needs(X,Y) :- prereq(Y,X).
needs(X,Y) :- prereq(Z,X), needs(Z,Y).
shares_prereq(X,Y)  :- prereq(Z,X), prereq(Z,Y), is_different(X,Y).
shares_prereq2(X,Y) :- is_different(X,Y), prereq(Z,X), prereq(Z,Y).
```

Give **all** the answers that Prolog returns for the following queries, in the correct order.

(a) `?- prereq(cs1100, X).`
(b) `?- needs(cs3100, X).`
(c) `?- needs(X, cs1100).`
(d) `?- needs(cs3100, cs3100).`
(e) `?- shares_prereq(cs3100, Y).`
(f) `?- shares_prereq2(cs3100, Y).` Explain the difference from (e) in one sentence.
(g) Define `grandprereq(X,Y)`, which holds when `Y` is a direct prerequisite of a direct prerequisite of `X`, and give all answers to `?- grandprereq(cs3100, Y).`

4. Lists and arithmetic.

   (a) Consider the program

   ```
   mys([],[]).
   mys([X],[X]).
   mys([X,Y|T],[Y,X|R]) :- mys(T,R).
   ```

   (i) Give all answers to `?- mys([1,2,3,4,5],L).`

   (ii) Give all answers to `?- mys(L,[a,b]).`

   (iii) Describe in one sentence what `mys` does.

   (b) Consider the program

   ```
   nth([X|_],0,X).
   nth([_|T],N,X) :- M is N-1, nth(T,M,X).
   ```

   Give all answers to the following queries. If a query raises an error or does not terminate, say so and explain why.

   (i) `?- nth([a,b,c],2,X).`

   (ii) `?- nth([a,b,c],5,X).`

   (iii) `?- nth([a,b,c],N,b).`

   (iv) `?- nth([a,b,c],N,X).`

   (v) `?- nth(L,1,x).`

   (c) Modify `nth` so that queries (iii) and (iv) of part (b) work, i.e. so that the index can be computed from the list and the element. Call the new predicate `nth2`. Does `?- nth2([a,b,c],2,X).` still work? What does `?- nth2(L,1,x).` do?

   (d) Recall `sum` from the lectures:

   ```
   sum([],0).
   sum([H|T],N) :- sum(T,M), N is M+H.
   ```

   What happens on the query `?- sum(X,3).`? Trace the execution far enough to justify your answer.

5. Rule order. Consider `len` from Lecture 17 with its two clauses swapped:

   ```
   len3([_|T],N) :- len3(T,M), N is M+1.
   len3([],0).
   ```

   (a) Give all answers to `?- len3([1,2],N).`

   (b) What happens on `?- len3(L,1).`? Compare with `?- len(A,2).` from Lecture 17, which returns `A = [_,_]` and then loops when asked for a second answer.

6. Cuts.

   (a) Consider the program

   ```
   p(1). p(2). p(3).
   q(X) :- p(X), X > 1, !.
   q(0).
   r(X) :- !, p(X).
   r(0).
   s(X) :- p(X), !, X > 1.
   s(0).
   ```

   Give all answers to the queries `?- q(X).`, `?- q(1).`, `?- q(0).`, `?- r(X).`, `?- r(0).`, `?- s(X).`, `?- s(0).` and `?- s(3).`

   (b) Consider the following attempt at a maximum predicate:

```
max(X,Y,Y) :- Y >= X, !.
max(X,_,X).
```

(i) What are the answers to `?- max(1,2,Z).` and `?- max(2,1,Z).`?

(ii) What is the answer to `?- max(1,2,1).`? Is it correct?

(iii) Fix the definition, keeping the cut, so that it is correct for every query with numeric first two arguments.

(c) For each of the following cuts, say whether it is red or green, with a one-line justification.

(i) The cut in the second clause of `split2` from the lectures: `split2([H|T],[H|L],R) :- H < 5, !, split2(T,L,R).`

(ii) The cut in `not(A) :- A, !, fail.`

(iii) The cut in the third clause of `remove_stutter` from the lectures:

```
remove_stutter([H,H|T],L) :- !, remove_stutter([H|T],L).
remove_stutter([X,Y|T],[X|L]) :- remove_stutter([Y|T],L).
```

(iv) The cut in `q` of part (a).

(d) Consider two definitions of list membership:

```
mem1(X,[X|_]) :- !.              mem2(X,[X|_]).
mem1(X,[_|T]) :- mem1(X,T).      mem2(X,[_|T]) :- mem2(X,T).
```

(i) Give a query with a variable in the *first* argument for which the two definitions return different answers, and give the answers.

(ii) Give a query with variables in the *second* argument for which they return different answers, and give the answers.

(iii) Can a ground query (no variables at all) distinguish them?

(e) What does the following predicate compute? Give the answers to `?- t(3,[1,2]).`, `?- t(1,[1,2]).` and `?- t(X,[1,2]).`

```
t(X,L) :- mem2(X,L), !, fail.
t(_,_).
```

7. Negation. Consider the program

```
student(ram). student(sita). student(arjun). student(meera).
registered(ram, cs3100).
registered(sita, cs3100).
registered(sita, cs3300).
registered(arjun, cs3300).
passed(ram, cs3100).
passed(sita, cs3300).
pending1(S,C) :- registered(S,C), not(passed(S,C)).
pending2(S,C) :- not(passed(S,C)), registered(S,C).
free1(S) :- student(S), not(registered(S,_)).
free2(S) :- not(registered(S,_)), student(S).
```

(a) Give all answers to `?- pending1(S,C).`

(b) Give all answers to `?- pending2(S,C).`

(c) Give all answers to `?- pending2(arjun,C).` and to `?- pending2(sita,C).` Explain why one of them works and the other does not, even though `sita` has a pending course.

(d) Give all answers to `?- free1(S).` and `?- free2(S).`

(e) Using quantifiers, explain in one or two sentences what `not(passed(S,C))` means when both variables are unbound, and state the recommendation from the lectures that `pending2` and `free2` violate.

(f) Give all answers to `?- student(X), not(X = sita).` and to `?- not(X = sita), student(X).`

(g) Define `all_passed(S)`, which holds when the student `S` has passed every course they are registered for (a student registered for nothing counts). Give all answers to `?- all_passed(S).`

8. Type inference for lists. Recall the type inference program from Lecture 21 (with the occurs check enabled):

```
lookup([(X,A)|_],X,A).
lookup([(Y,_)|T],X,A) :- X \== Y, lookup(T,X,A).

type(_,u,unit).
type(G,app(M,N),B)      :- type(G,M,A -> B), type(G,N,A).
type(G,lam(X,M),A -> B) :- type([(X,A)|G],M,B).
type(G,fst(M),A)        :- type(G,M,A * _).
type(G,snd(M),B)        :- type(G,M,_ * B).
type(G,pair(M,N),A * B) :- type(G,M,A), type(G,N,B).
type(G,var(X),A)        :- lookup(G,X,A).
type(_, X, int)         :- integer(X).
type(_, true, bool).
type(_, false, bool).
type(G, A + B, int)     :- type(G,A,int), type(G,B,int).
type(G, A - B, int)     :- type(G,A,int), type(G,B,int).
type(G, A < B, bool)    :- type(G,A,int), type(G,B,int).
type(G, ite(A,B,C), T)  :- type(G,A,bool), type(G,B,T), type(G,C,T).
type(Term,Type) :- type([],Term,Type).
```

We extend the checker with lists. The new terms and their encodings are

M, N	::=	...		
	|	nil	(empty list)	`nil`
	|	cons M N	(cons)	`cons(M,N)`
	|	hd M	(head)	`hd(M)`
	|	tl M	(tail)	`tl(M)`
	|	isnil M	(emptiness test)	`isnil(M)`

and the new type list A is encoded as `list(A)`. The typing rules are

$$\frac{}{\Gamma \vdash \mathsf{nil} : \mathsf{list}\ A}\ (nil) \qquad \frac{\Gamma \vdash M : A \quad \Gamma \vdash N : \mathsf{list}\ A}{\Gamma \vdash \mathsf{cons}\ M\ N : \mathsf{list}\ A}\ (cons)$$

$$\frac{\Gamma \vdash M : \mathsf{list}\ A}{\Gamma \vdash \mathsf{hd}\ M : A}\ (hd) \qquad \frac{\Gamma \vdash M : \mathsf{list}\ A}{\Gamma \vdash \mathsf{tl}\ M : \mathsf{list}\ A}\ (tl)$$

$$\frac{\Gamma \vdash M : \mathsf{list}\ A}{\Gamma \vdash \mathsf{isnil}\ M : \mathsf{bool}}\ (isnil)$$

(a) Write the five `type/3` clauses for the new constructs.

(b) Using your clauses, give the answer of each query (the inferred type, or `false`). Write type variables as `A`, `B`, ...

(i) $\lambda x.\,\mathsf{cons}\ x\ \mathsf{nil}$: `?- X = var(x), type(lam(x,cons(X,nil)),T).`

(ii) cons 1 (cons true nil): `?- type(cons(1,cons(true,nil)),T).`

(iii) $\lambda l.\,\mathsf{if}\ \mathsf{isnil}\ l\ \mathsf{then}\ 0\ \mathsf{else}\ \mathsf{hd}\ l$: `?- L = var(l), type(lam(l,ite(isnil(L),0,hd(L))),T).`

(iv) $\lambda l.\,\mathsf{cons}\ (\mathsf{hd}\ l)\ (\mathsf{tl}\ l)$: `?- L = var(l), type(lam(l,cons(hd(L),tl(L))),T).`

9. De Bruijn indices. Lecture 8 mentioned how compilers avoid the whole business of fresh names during substitution in λ-calculus: *de Bruijn indices*. A bound variable is replaced by the number of binders between its occurrence and the λ that binds it, and the binders carry no names at all. So $\lambda x.\,x$ becomes

$\lambda.\,0$, $\lambda x.\lambda y.\,x$ becomes $\lambda.\lambda.\,1$, $\lambda x.\lambda y.\lambda z.\,(x\ z)\ (y\ z)$ becomes $\lambda.\lambda.\lambda.\,(2\ 0)\ (1\ 0)$, and $(\lambda x.x)(\lambda y.y)$ becomes $(\lambda.\,0)(\lambda.\,0)$. Two terms are α-equivalent exactly when their de Bruijn forms are identical, with no renaming needed. In this question, we will write a Prolog program to compute De Bruijn indices.

λ-terms are represented as in Question 8: `var(x)`, `lam(x,M)` and `app(M,N)`. De Bruijn terms are represented with `var/1` (now holding a number), `lam/1` (no name) and `app/2`, and a free variable, which no binder binds, with `free/1`. Writing $D(T)$ for the de Bruijn form of T:

λ-term	**de Bruijn term**
a bound variable with k binders between it's occurrence and its own binder	`var(k)`
a free variable x	`free(x)`
$\lambda x.\,T$	`lam(D(T))`
$T_1\ T_2$	`app(D(T1),D(T2))`

For example:

- $D(\lambda x.\lambda y.\,x\ y)$ is `lam(lam(app(var(1), var(0))))`;
- $D(\lambda x.\,x\ y)$ is `lam(app(var(0), free(y)))`.

(a) Give the de Bruijn forms of the following terms.

(i) `lam(x, lam(y, lam(z, app(app(var(x), var(z)), app(var(y), var(z))))))`

(ii) `lam(x, app(lam(y, app(var(y), var(x))), var(z)))`

(iii) `app(var(x), lam(x, var(x)))`

(iv) `lam(x, lam(y, lam(x, var(x))))`

(b) Define the predicate `de_bruijn(T,D)`, which holds if `D` is the de Bruijn form of the λ-term `T`. The predicate should have exactly one answer for every term.

Hint: as in the type inference of Lecture 21, carry an environment, here simply the list of the binders in scope, innermost first; define `de_bruijn` as a ternary predicate first, which takes the environment as an extra argument. Define a predicate `position(L,X,N)` which holds if `N` is the first index at which `X` appears in `L`, with indexing beginning from 0. Use this `position` predicate in the definition of `de_bruijn`.

10. Programming problems. Define the following predicates. You may define auxiliary predicates, and you may use `append/3` and `member/2`. Make sure your definitions give exactly the answers shown, with no spurious extra answers, for the example queries.

(a) `range(M,N,L)`: `L` is the list of integers from `M` to `N` inclusive, in increasing order; it is `[]` if `M > N`. E.g. `range(2,5,[2,3,4,5])`, `range(5,2,[])`, `range(3,3,[3])`.

(b) `split_at(N,L,F,B)`: `F` is the list of the first `N` elements of `L` and `B` is the rest. It should fail if `L` has fewer than `N` elements. E.g. `split_at(2,[a,b,c,d],[a,b],[c,d])`, `split_at(0,[a,b],[],[a,b])`. Then use it to define `rotate(L,N,R)`: `R` is `L` rotated to the left by `N` positions, e.g. `rotate([a,b,c,d],1,[b,c,d,a])` and `rotate([a,b,c,d],3,[d,a,b,c])`.

(c) `prefix_sums(L,R)`: the i-th element of `R` is the sum of the first i elements of `L`, e.g. `prefix_sums([1,2,3,4],[1,3,6,10])`. Use an accumulator so that the definition is tail recursive.

(d) `rle(L,R)` (run-length encoding): `R` lists each maximal run of equal adjacent elements of `L` as a pair `X-N`, e.g. `rle([a,a,b,c,c,c],[a-2,b-1,c-3])`, `rle([a,b,a],[a-1,b-1,a-1])`, `rle([],[])`.

(e) `remove_dups(L,R)`: `R` is `L` with every element kept only at its *first* occurrence, e.g. `remove_dups([1,2,1,3,2],[1,2,3])`.

(f) `evens_odds(L,E,O)`: `E` and `O` are the even and the odd integers of `L`, each in their original order, e.g. `evens_odds([1,2,3,4,5],[2,4],[1,3,5])`. (`X mod 2` is the remainder of `X` divided by 2.)

(g) Binary trees are represented as in the lectures: `leaf` is the empty tree and `tree(L,X,R)` has left subtree `L`, value `X` and right subtree `R`.

(i) `tree_height(T,H)`: `H` is the height of `T` (`leaf` has height 0, a single node has height 1).

(ii) `mirror(T,M)`: `M` is the mirror image of `T`, e.g.

```
mirror(tree(tree(leaf,b,leaf),a,tree(leaf,c,leaf)),
       tree(tree(leaf,c,leaf),a,tree(leaf,b,leaf)))
```

holds.

(iii) `bst_insert(X,T,T2)`: `T2` is the binary search tree obtained by inserting the number `X` into the binary search tree `T` (values less than or equal to the node's value go left, larger ones right). E.g. inserting 5, 3, 8, 4 in this order into `leaf` gives `tree(tree(leaf,3,tree(leaf,4,leaf)),5,tree(leaf,8,leaf))`.